BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 304 (January 2024)

Digital Electronics Laboratory

Final Project Report

Section: A2 Group: 08

Mechanical Braille Cell using FPGA board

Course Instructors:

Dr. Sajid Muhaimin Choudhury, Associate Professor
H M Tahmeed-Ul-Islam, Part-time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: Anha **Full Name: Sumaiya Tabassum Anha** **Student ID: 2006056**	**Signature:** Sajin **Full Name: Mohammed Irtisam Sajin** **Student ID: 2006057**
Signature: Asmaul Husna Pushpita **Full Name: Asmaul Husna Pushpita** **Student ID: 2006058**	**Signature:** Zubaer H. Nomi **Full Name: Md. Zubaer Hossain** **Student ID: 2006061**

Table of Contents

1 Abstract

This project focused on the development of a mechanical Braille cell designed to enhance accessibility for visually impaired individuals. The primary objective is to create a device that combines simplicity in structure with ease of use, utilizing an FPGA board to control the mechanism. By leveraging the advanced capabilities of the FPGA, the project aims to deliver a reliable and user-friendly Braille cell that could potentially improve the quality of life for its users.

2 Introduction

A refreshable mechanical Braille display is a pivotal technological advancement that enables blind and visually impaired individuals to read text through tactile feedback. This device functions by translating digital text into Braille characters, which are represented by patterns of raised pins on a surface. Each pin corresponds to a cell in a grid, and these pins can be individually raised or lowered to form Braille characters.

The primary interaction with the device involves users inputting text via a connection to a digital device. The Braille display then converts this text into tactile Braille characters by manipulating the pins accordingly. Users read the text by feeling these raised pins with their fingertips, allowing them to discern letters, numbers, and symbols.

The conversion from input text characters to Braille characters is done using an FPGA board. It generates necessary signals for raising and lowering of the pins in the Braille cell and thus allowing the user to read the text using the Braille patterns.

3 Design

3.1 Problem Formulation

3.1.1 Identification of Scope

The project is to be implemented without the use of any microcontrollers like Arduinos. The design has to be simple and the implementation should be done using logic gates and/or basic ICs. CPLD or FPGA boards can be used as well.

3.1.2 Literature Review

Braille patterns have been used to aid those who are visually impaired for a long time. The patterns are quite universal, which have been showed in the image below

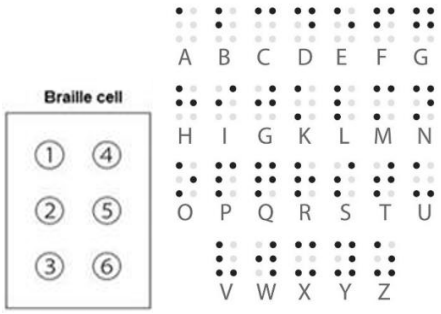


Figure 1: Braille Patterns

The corresponding ASCII codes for the characters range from 65 (for A) to 90 (for Z).

3.1.3 Formulation of Problem

The entire project can roughly be divided into the following major problems that are to be addressed:

- Conversion from ASCII codes to Braille patterns.
- Generation of corresponding signals for mechanical control.
- Mechanical control of the braille pins.

3.2 Design Method

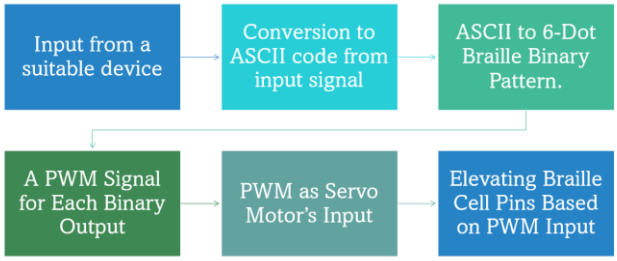


Figure 2: Overview of the project

The project is to be implemented using an FPGA board (Altera University Program UP2 Education Kit). Verilog is used for coding the entire program. Besides, knowledge about PWM signal generation and the hardware part (Servo Motors and Timer Circuit) is required.

The EPF10K70 device of the FPGA board is utilized for the project. For the coding, simulation and implementation of our design on the board, Quartus II software is used.

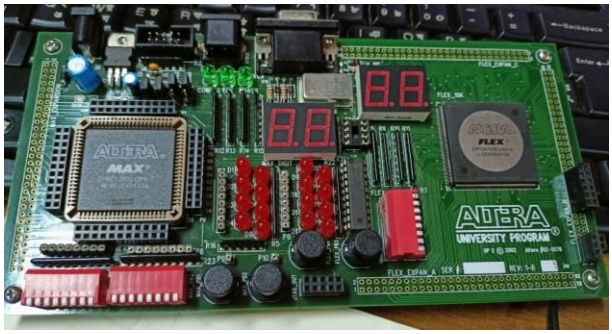


Figure 3: Altera University Program UP2 Education Kit

4 Implementation

4.1 Description

Input of Characters: We utilized the eight switches present on the FLEX 10K section of the FPGA board for providing the 8 bit binary representation of the ASCII Character codes as input.

Table 6. FLEX_SW1 Pin Assignments

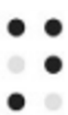
Switch	FLEX 10K Pin
FLEX_SWITCH-1	41
FLEX_SWITCH-2	40
FLEX_SWITCH-3	39
FLEX_SWITCH-4	38
FLEX_SWITCH-5	36
FLEX_SWITCH-6	35
FLEX_SWITCH-7	34
FLEX_SWITCH-8	33

Figure 3: Pin arrangement of the eight Switches.

The corresponding 8-bit input variables for the character are assigned to the FLEX 10K Pins and thus using the switches, characters could be provided as input. For our implementation, we are providing the 8-bit Binary representation of the character's ASCII code.

Conversion to ASCII Code: Since, we are directly providing the ASCII code to the board, this step is skipped for our implementation. But if any other input method was used, the input signal would first have to be converted into ASCII code and then continue with the remaining steps

ASCII to 6-bit Braille Binary pattern: The Braille pattern consist of 6 pins either elevated (ON) or lowered (OFF). Figure 1 shows the Braille cell and the Braille patterns for the characters. The pins have been numbered from 1 to 6 and a 6-bit binary pattern is utilized for representing the pins. Each bit would be assigned either 1 (for ON) and 0 (for OFF) based on the patterns as depicted in Figure 1 and also in the following table:

Character	ASCII Code	ASCII Binary Code (8-bit)	Braille Binary Pattern (6-bit)	Braille Pattern
A	65	01000001	100000	
B	66	01000010	110000	
C	67	01000011	100100	
D	68	01000100	100110	
E	69	01000101	100010	
F	70	01000110	110100	
G	71	01000111	110110	
H	72	01001000	110010	
I	73	01001001	010100	
J	74	01001010	010110	
K	75	01001011	101000	
L	76	01001100	111000	
M	77	01001101	101100	
N	78	01001110	101110	

O	79	01001111	101010	
P	80	01010000	111100	
Q	81	01010001	111110	
R	82	01010010	111010	
S	83	01010011	011100	
T	84	01010100	011110	
U	85	01010101	101001	
V	86	01010110	111001	
W	87	01010111	010111	
X	88	01011000	101101	
Y	89	01011001	101111	
Z	90	01011010	101011	

The next step was to come up with the necessary equations that would take the 8-bit ASCII code and would provide the 6-bit Braille Binary pattern as output. For this purpose, we used a K-map solver and came up with 6 equations for the 6 output pins.

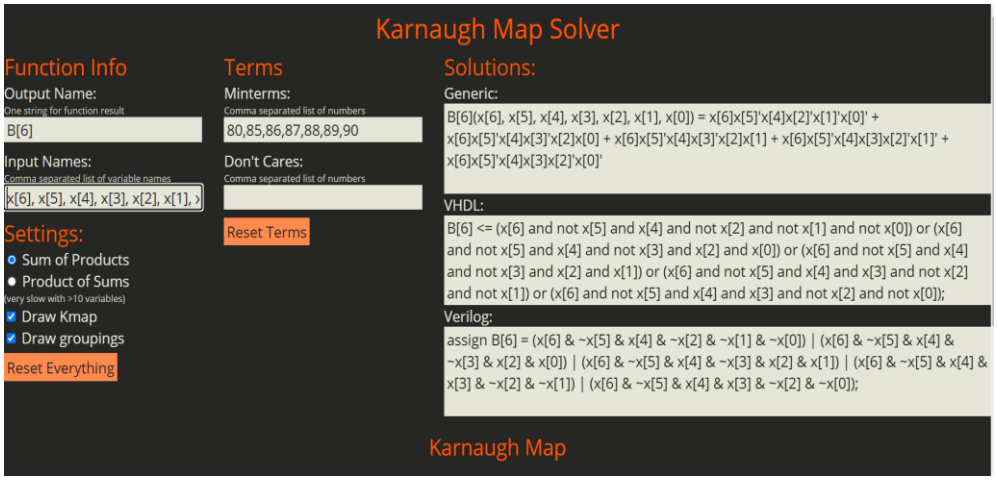


Figure 4: Online K-map solver for equation generation.

Figure 4 shows the generation of the equation for the output of pin 6 (B[6]). Similarly, we generated the remaining 5 equations and used the generated Verilog code in our main code file.

Generation of PWM signals: For elevating and lowering the braille pins, we are using SG90 Servo motors. The PWM input that is given to the servo is responsible for adjusting the rotation angle.

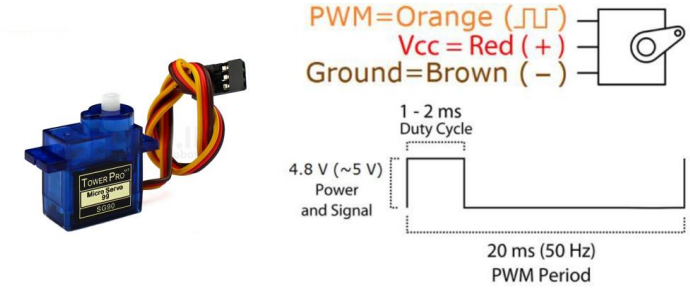


Figure 5: Servo motor PWM signal

Figure 5 provides us the information about the required PWM signal. The time period of the signal should be around 20ms. The HIGH time for the signal can be varied from 1 to 2ms. 1ms HIGH time corresponds to 0 degree position and 2ms corresponds to 180 degree position. The angles change linearly from 0 to 180 degree between these two boundary HIGH time values (1ms to 2ms). So, by changing the HIGH time the servo can be rotated to any angle from 0 to 180 degree. For our project we will be needing two angles: 0 degree for OFF state of Braille pin and 90 degrees for the ON state. For 0 degree the Pulse HIGH time is 1ms and for 90 degrees the HIGH time is calculated using the following equation: $\boldsymbol{T = \frac{90}{180} + 1 = 1.5ms}$

To implement this using Verilog, we first needed a high frequency clock signal. The FPGA board has an internal clock signal of 25.175 MHz frequency. But the issue with this clock signal is that the peak voltage is too low, around 200mV.

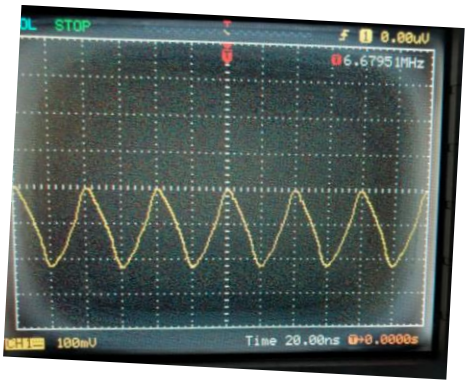


Figure 6: Internal clock signal of FPGA board

The peak voltage is not sufficient to generate our desired PWM signal. Hence, we designed a separate clock signal generator using 555 Timer IC. The circuit diagram along is shown below:

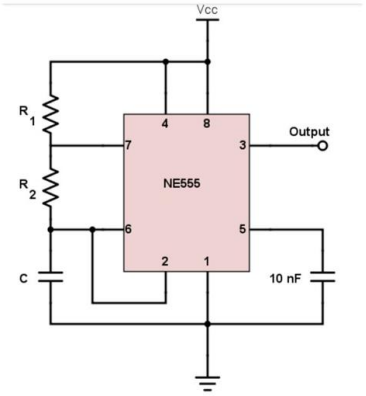


Figure 7: 555 Timer IC Clock generator circuit diagram

The parameters are selected based on the below equations:

$$\boldsymbol{Frequency = \frac{1}{Time\ Period} = \frac{1.44}{(R_1 + 2R_2) * C}}$$

$$\boldsymbol{Duty\ Cycle = \frac{R_1 + R_2}{R_1 + 2R_2}}$$

$$\boldsymbol{We\ used\ R_1 = 20K\Omega, R_2 = 120K\Omega, C = 0.1nF}$$
$$\boldsymbol{Hence, Frequency = 55.384\ KHz, and\ duty\ cycle = 54.85\%}$$

We implemented the circuit in hardware and obtained a clock signal of 51.282KHz and duty cycle slightly above 50%. The crucial part is that the peak voltage is around 4V, which is high enough for our PWM signal generation.

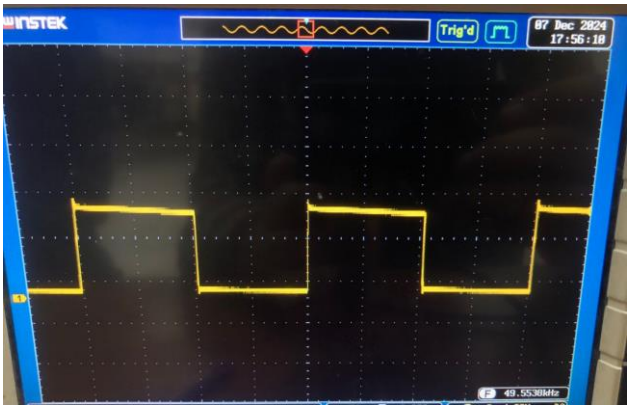


Figure 8: Clock signal from Timer IC circuit

For the PWM signal generation, we calculated the number of clock cycles required to complete 20ms time period and also the number of cycles required for min Pulse time (1ms for 0 degree) and max Pulse time (1.5ms for 90 degrees). These equations are as following:

$$\textbf{For 20 ms time period, } Counter_{max} = \frac{20ms}{\textbf{Clock signal time period}} = \frac{\textbf{Clock signal frequency}}{50}$$

$$\textbf{For min Pulse ON time, } PULSE_{MIN} = \frac{1ms}{\textbf{Clock signal time period}} = \frac{\textbf{Clock signal frequency}}{1000}$$

$$\textbf{For max Pulse ON time, } PULSE_{MAX} = \frac{1.5}{\textbf{Clock signal time period}} = \frac{\textbf{Clock signal frequency}}{666.6667}$$

When the number of clock cycles equals to $Counter_{max}$ a new PWM signal starts and the number of clock cycles is reset to 0. If a Braille pin is ON, we want to rotate the corresponding servo to 90 degrees. Hence, we need the Pulse signal to be high for 1.5ms. Thus, when the number of cycles is less than $PULSE_{MAX}$ the output will be high, otherwise the output will be low. The resulting the PWM output is shown in the timing diagram below:

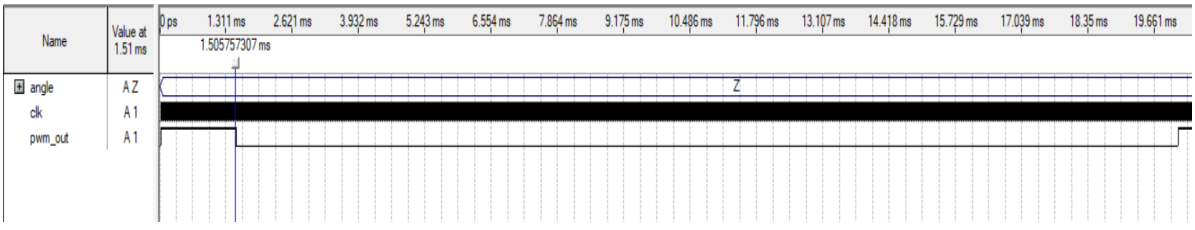


Figure 9: Timing diagram of Pulse signal for 90 degree rotation

The time period of this signal is around 20ms and the Pulse HIGH time is around 1.5ms. Similarly, for the rotation angle to be 0 degree, the Pulse HIGH time is to be around 1ms. That is, when the number of clock cycles is less than $PULSE_{MIN}$ the output will be high, otherwise the output will be low.

Mechanical Control of Braille Cell: We then implemented the mechanical part of our project. The circuit diagram is shown below:

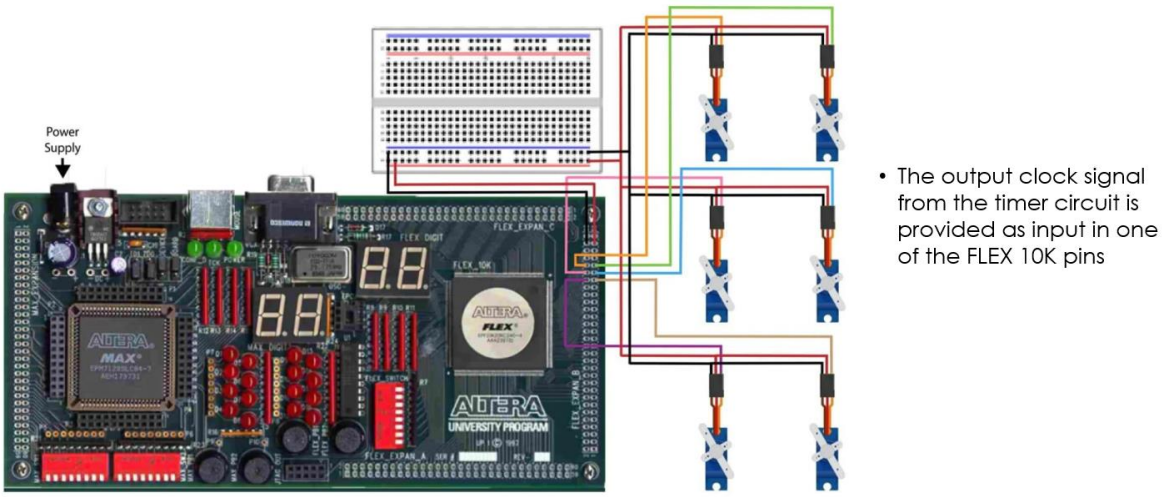


Figure 10: Circuit diagram

The timer IC circuit is powered by a 8V DC voltage for gaining a high enough peak voltage. The output of this circuit is provided as input to one of the assigned I/O pins of the board. The Servo motors are powered by a 4.5V DC voltage. The 6 PWM pins of the 6 motors are assigned to 6 I/O pins of the board. Each Servo motors will receive individual PWM signals, such that HIGH time will be either 1ms (when the pin needs to be lowered) or 1.5ms (when the pin needs to be elevated).

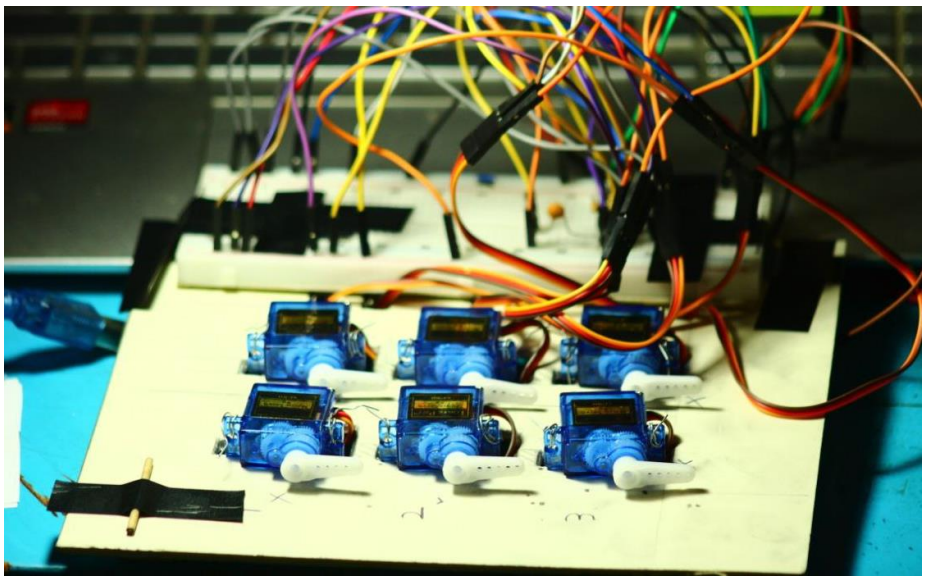


Figure 11: Braille Cell

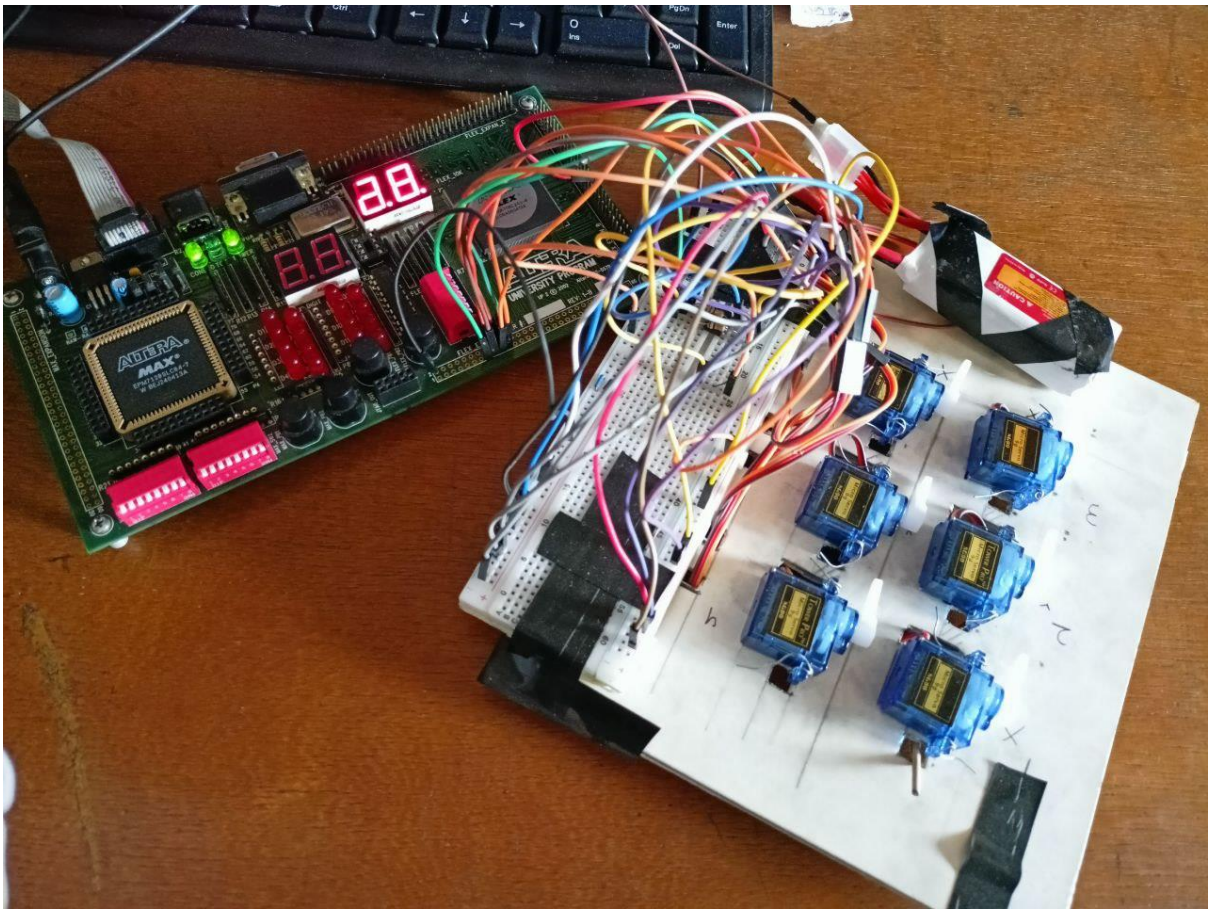


Figure 12: Entire Setup

```
module braille(x,B,clk,pwm_out);
input [0:6]x;
output reg [1:6]B;
input wire clk;
output reg [1:6] pwm_out;


 ///PWM Code Starts
 parameter CLOCK_FREQ = 51282;
    parameter PWM_FREQ = 50;
    localparam COUNTER_MAX = CLOCK_FREQ / PWM_FREQ;
    localparam PULSE_MIN = CLOCK_FREQ / 1000;
    localparam PULSE_MAX = CLOCK_FREQ / 666.6667;
    reg [31:0] counter = 0;
          reg [31:0]
p1=PULSE_MIN,p2=PULSE_MIN,p3=PULSE_MIN,p4=PULSE_MIN,p5=PULSE_MIN
,p6=PULSE_MIN;
    always @(posedge clk) begin
        if (counter < COUNTER_MAX) begin
            counter <= counter + 1;
        end else begin
            counter <= 0;
        end
    B[1] <= ((x[0] & ~x[1]) & ((~x[2] & x[4]) | (~x[2] & x[5] &
x[6]) |
    (~x[3] & ~x[5] & x[6]) | (~x[3] & x[5] & ~x[6]) | (x[2] &
~x[4] & ~x[5]) | (x[2] & ~x[4] & ~x[6]) | (~x[2] & x[3] & ~x[5]
& ~x[6])));

    B[2] <= ((x[0] & ~x[1]) & ((~x[3] & x[4] & x[5]) |
    (~x[2] & x[3] & ~x[4] & ~x[5]) | (~x[2] & x[3] & ~x[5] &
~x[6]) | (x[2] & ~x[3] & ~x[4]) | (x[2] & ~x[3] & ~x[6]) |
    (~x[2] & ~x[4] & x[5] & ~x[6])));

    B[3] <= ((x[0] & ~x[1]) & ((~x[2] & x[3] & x[5] & x[6]) |
    (~x[2] & x[3] & x[4]) | (x[2] & ~x[3] & ~x[4]) | (x[2] &
~x[3] & ~x[5]) | (x[2] & ~x[3] & ~x[6]) | (x[2] & ~x[4] & ~x[5])
|
    (x[2] & ~x[4] & ~x[6])));

    B[4] <= ((x[0] & ~x[1]) & ((~x[3] & x[5] & x[6]) |
    (~x[2] & x[3] & x[5] & ~x[6]) | (~x[2] & x[3] & ~x[5] &
x[6]) | (x[2] & ~x[4] & ~x[5]) | (~x[2] & ~x[3] & x[4] & ~x[6])
|
    (~x[3] & x[4] & ~x[5] & ~x[6])));

    B[5] <= ((x[0] & ~x[1]) & ((~x[2] & x[3] & ~x[4] & ~x[6]) |
    (x[2] & ~x[4] & ~x[5] & x[6]) | (x[2] & ~x[4] & x[5] &
~x[6]) | (~x[3] & x[4] & ~x[5] & ~x[6]) |
    (~x[3] & x[4] & x[5] & x[6]) | (~x[2] & ~x[3] & x[4] &
~x[5]) | (~x[2] & x[3] & x[4] & x[5])));

    B[6] <= ((x[0] & ~x[1]) & ((x[2] & ~x[3] & x[4] & x[6]) |
    (x[2] & ~x[3] & x[4] & x[5]) | (x[2] & x[3] & ~x[4] & ~x[5])
| (x[2] & x[3] & ~x[4] & ~x[6])));
```

```
if(B[1]==1 && p1<PULSE_MAX)p1<=p1+1;
    else if(B[1]==0 && p1>PULSE_MIN)p1<=p1-1;
    pwm_out[1] <= (counter < p1) ? 1 : 0;

    if(B[2]==1 && p2<PULSE_MAX)p2<=p2+1;
    else if(B[2]==0 && p2>PULSE_MIN)p2<=p2-1;
    pwm_out[2] <= (counter < p2) ? 1 : 0;

    if(B[3]==1 && p3<PULSE_MAX)p3<=p3+1;
    else if(B[3]==0 && p3>PULSE_MIN)p3<=p3-1;
    pwm_out[3] <= (counter < p3) ? 1 : 0;

    if(B[4]==1 && p4<PULSE_MAX)p4<=p4+1;
    else if(B[4]==0 && p4>PULSE_MIN)p4<=p4-1;
    pwm_out[4] <= (counter < p4) ? 1 : 0;

    if(B[5]==1 && p5<PULSE_MAX)p5<=p5+1;
    else if(B[5]==0 && p5>PULSE_MIN)p5<=p5-1;
    pwm_out[5] <= (counter < p5) ? 1 : 0;

    if(B[6]==1 && p6<PULSE_MAX)p6<=p6+1;
    else if(B[6]==0 && p6>PULSE_MIN)p6<=p6-1;
    pwm_out[6] <= (counter < p6) ? 1 : 0;
    end
endmodule
```

Source Code for the main program

5 Design Analysis and Evaluation

5.1 Novelty

Our design provides an easy and efficient way for visually impaired people to read texts. The design is simple and adaptable to any input methods with slight modifications. The braille cell refreshes as characters are changed and thus can be used in many applications.

5.2 Design Considerations

Our design was made addressing Public Health and Safety Concerns, Environmental factors, Cultural and Societal Needs. Visually impaired people can easily use this device to read texts without health risks. The device is simple and the pins are not sharp which enables easy and safe reading for the users. Environmental factor was taken into account and minimal electronics and plastic wastes were used. The device has a big Cultural and Societal Impact as it creates opportunities for people who are willing to read but not able to due to disability. Thus, it promotes equal access to education, employment, and other opportunities.

5.3 Investigations

5.3.1 Design of Experiment

The setup was made as shown in Figure 12. Next the cell was tested by providing the inputs of each of the 26 characters and the outputs were observed.

5.3.2 Results and Analysis

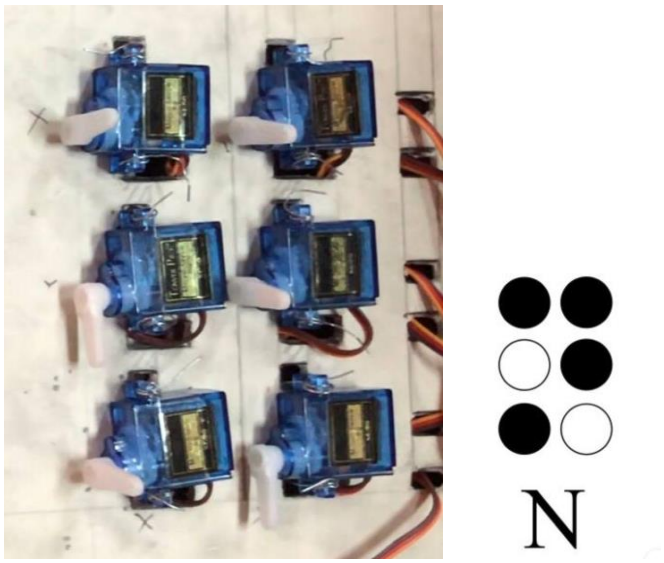


Figure 13: Output for character “N”

The output for character “N” shows expected result as the pattern of our braille cell matches the expected Braille pattern for “N” (black dots mean elevated and white dots mean lowered pins).

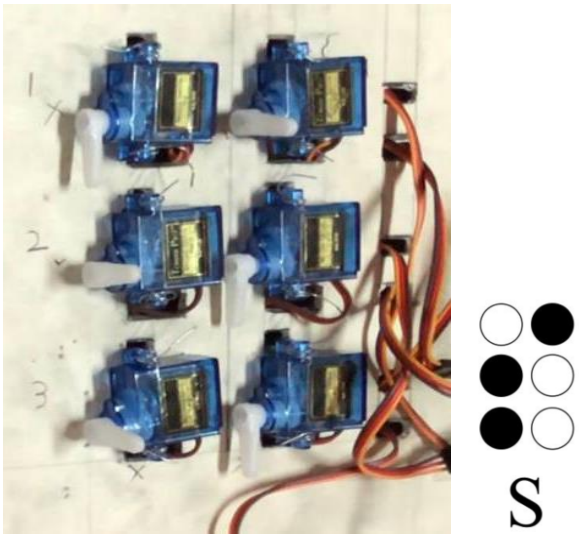


Figure 14: Output for character “S”

The output for character “S” also shows expected result. The outputs for the remaining characters were observed and showed expected result. Thus, we can conclude that the Braille cell is performing as it was intended to.

5.4 Limitations of Tools

Since, we were not able to use any microcontrollers, we had to rely on the FPGA board that we had to use while being present in the designated Laboratory for the course. Thus, the time for the work was limited by the availability of the laboratory schedule.

5.5 Ethical Issues

We worked diligently to ensure that our project adheres to ethical standards throughout its development and implementation. This includes considering the social implications of our technology. By fostering transparency and accountability, we aim to create a positive impact that aligns with ethical practices and promotes the well-being of all the stakeholders involved.

6 Reflection on Individual and Team work

6.1 Individual Contribution of Each Member

- 2006056: Braille Binary Pattern Generation, Servo rotation using PWM
- 2006057: Clock signal Generation, Servo rotation using PWM
- 2006058: Braille Binary Pattern Generation, Servo rotation using PWM
- 2006061: Clock signal Generation, Servo rotation using PWM

6.2 Mode of Team Work

- Online Meetings: We utilized online platforms for regular meetings, allowing team members to connect regardless of location. This facilitated real-time discussions, brainstorming sessions, and quick decision-making, ensuring everyone stayed aligned on project goals.
- Offline Meetings: In addition to virtual gatherings, we held offline meetings to foster deeper connections and engage in more interactive discussions. These face-to-face sessions were instrumental in building rapport, enhancing communication, and collaboratively addressing complex issues.
- Laboratory meetings: As we required the FPGA board provided by the laboratory, we conducted our implementations and testing there.

6.3 Diversity Statement of Team

Our team is committed to fostering a diverse and inclusive environment where every member feels valued and empowered. We recognize that diverse perspectives enhance creativity and drive innovation in our project. By collaborating across various backgrounds and experiences, we harness our collective strengths to tackle challenges effectively.

6.4 Log Book of Project Implementation

Date	**Milestone achieved**	**Individual Role** (last two digits of student IDs are used)	**Comments**
04/09/24	Formation of group		The project was formed and was handed the task for finding project ideas.
21/09/24	Project Proposal	56 and 58 prepared the project proposal containing the initial project idea	
22/09/24	Project Assigned		The course teachers accepted and thus a project was assigned to the group
28/10/24	Braille Pattern Generation	56 and 58 provided the equations and code for this task	
03/11/24	Project Update presentation	56, 57, 58 and 61 presented the project update to the course teachers	
08/11/24	Servo PWM generation and rotation test	57 and 61 made the first initial test for the Servo rotation	
12/11/24	Clock signal generation	57 and 61 implemented the timer IC circuit for clock signal generation	
20/11/24	Final implementation of the circuit	58 and 61 made the Braille cell structure using the servo motors 56 and 57 designed the connection between the braille cell and the FPGA board	
07/12/24	Successful output	All the members worked to successfully ensure the output of the circuit	After attempts over a couple of days, the final output from the circuit was obtained.
15/12/24	Final Project Demonstration	All the members demonstrated the project to the course teachers	

7 Executive Summary

The aim of the project was to develop a simple and easy to use mechanical Braille cell. Our primary focus was to design a minimal structure and ensure that it provides an efficient way to convert text into braille patterns for the visually impaired people. We achieved expected outcomes which can have huge cultural and social impacts.

8 Bill of Materials

Components	No of units	Unit Price (Tk)	Total Price (Tk)
FPGA Board (Supplied by Lab)	1	-	-
SG90 Servo Motors	6	100	600
Jumper Wires	1 set	100	100
Breadboard	1	150	150
555 Timer	1	15	15
Resistors & Capacitors	4	-	10
		Total	**875**

9 Future Work

- The circuit designed in a way such that any input methods can be used. One only needs to ensure the conversion of the input signal into ASCII code and the remaining part can be used as it is
- The project can be designed in a smaller form factor
- Integration of more symbols as inputs for the braille cell.

10 References

- Braille Code Conversion using Verilog: https://www.ijresm.com/Vol.3_2020/Vol3_Iss5_May20/IJRESM_V3_I5_249.pdf
- Karnaugh Map Solver: https://www.charlie-coleman.com/experiments/kmap/
- 555 Timer Astable Oscillator Circuit: https://www.allaboutcircuits.com/tools/555-timer-astable-circuit/